

SolarFit — Product Requirements Document (PRD)

Regional Solar Generation & Revenue Simulator (for Installers)

Introduction to AI Programming · Assignment 4

Date: 2026-06-14

Submission Information

- **GitHub Repository:** <https://github.com/knameds-cmd/Assignment4>
- **Live Deployment (Vercel):** <https://assignment4-51gw.vercel.app>
- **Author:** knameds (knameds@gmail.com)
- **Tech Stack:** Next.js 15 (App Router), React 19, CSS Modules, browser localStorage
- **Document contents:** Product Requirements Document (PRD) + AI-Assisted Development Report

The full source code is available in the GitHub repository above. This document focuses on the PRD and the AI development report.

1. Product Overview

SolarFit is a web-based simulator that helps solar power installers instantly estimate a candidate site's **annual generation, expected revenue, payback period, and CO₂ reduction**, and present those figures to clients on the spot.

Small installers must repeat these calculations for every new site. Because regional solar resource, system losses, and SMP/REC prices change each time, they end up rebuilding spreadsheets and cannot answer client questions quickly. SolarFit automates this repetitive work with standard estimation formulas, so a single set of inputs produces all the key metrics needed for a feasibility check on one screen.

- **Problem solved:** Repetitive per-site generation/revenue calculations and the inability to answer on the spot.
- **Core value:** Provides a credible estimate in about 30 seconds and compares multiple sites.
- **Realism (curtailment):** In regions where grid constraints force frequent curtailment (e.g., Jeju), part of the generation cannot be sold; this is reflected through a "curtailment rate" so revenue is estimated conservatively.
- **Data:** Regional solar resource, historical PV generation, and curtailment use representative values referenced from Korean public statistics (KMA, Korea Energy Agency, Korea Power Exchange / KPX), with sources and limitations clearly labeled.

- **Web advantage:** A calculation can be shared as a URL link, so unlike a spreadsheet the installer can send a client the exact same result screen instantly.
- **Constraint:** Runs entirely in the browser (localStorage) with no backend, database, or paid API.

2. Target Users

User	Needs / Pain points	Usage context
Small solar installers	Want to quickly estimate generation/revenue per site and propose it. Repetitive spreadsheet work is tedious.	After a site survey, just before/during a client consultation
Solar sales representatives	Want to give an approximate return immediately and build comparison material.	Phone / in-person consultations
Building owners considering solar	Want an intuitive sense of the benefit for their region and building size.	Information-gathering stage

The primary target is the **small solar installer**, who needs both the simplicity to use the tool with no training and the flexibility to adjust assumptions to on-site conditions.

3. Project Goals

- **G1.** Calculate annual generation and expected revenue within 3 seconds from region and system inputs.
- **G2.** Let users compare and sort the generation potential of all 17 provinces/metropolitan cities on one screen.
- **G3.** Allow saving and comparing estimates for multiple sites without sign-up or a server.
- **G4.** Disclose every formula and assumption transparently so users can judge the reliability of results.
- **G5.** Provide a consistent experience on both mobile and desktop, and deploy on Vercel.

4. Core User Scenario

Situation — Mr. Kim, a solar installer working in Naju (Jeollanam-do), receives a phone call from a client asking, "How much will I earn per year if I put 100 kW on my roof?"

1. Mr. Kim opens the SolarFit **revenue simulator**.
2. He selects the region **Jeollanam-do** and a system capacity of **100 kW**.
3. He keeps the default SMP price (KRW 130) and REC weight (1.2), and adjusts only the performance ratio to match site conditions.
4. The screen immediately shows about **110,960 kWh/year of generation, ~KRW 23.7M annual revenue, a payback period of ~5.9 years, and 51 tons of CO₂ reduction**.
5. He names the estimate "Naju Hanbit Village rooftop" and **saves** it.

6. He saves other candidate sites the same way, then compares the highest-revenue and shortest-payback estimates on the **Saved Estimates** page to advise the client.

5. Feature List

Feature	Description	Priority
Revenue simulator	Real-time calculation of generation/revenue/payback from region, capacity, performance ratio, SMP/REC prices	Must-have
Regional solar resource data	Built-in average daily generation hours for all 17 provinces/cities	Must-have
Save/delete estimates (localStorage)	Save and manage calculation results in the browser	Must-have
Curtailement adjustment	Adjust "sellable generation" and revenue by a regional curtailement rate (reflecting grid constraints such as Jeju)	Should-have
Regional comparison (sort/filter/search)	Region-group filter, name search, generation sort, bar visualization	Should-have
Generation-potential map	Choropleth map of 17 provinces; hover/click highlights a region and syncs two-way with the comparison table	Should-have
Historical / monthly generation data	Visualize monthly generation pattern and national annual PV generation trend from public statistics	Should-have
Shareable result link	Encode inputs into a URL and copy it, so a client opens the exact same result screen (a web-only capability)	Should-have
Estimate comparison highlight	Automatically flag the highest-revenue and shortest-payback estimates	Should-have
Environmental-impact conversion	Convert to CO ₂ reduction and equivalent number of trees	Should-have
Methodology / disclaimer disclosure	Disclose formulas, assumptions, and data sources	Should-have
Export estimate to PDF	Output a saved estimate as a PDF proposal	Nice-to-have
Live irradiance API integration	Connect precise per-site irradiance data	Nice-to-have

6. Page Structure

- **Home (/)** — Service overview, feature summary, how-to-use, data sources
- **Simulator (/simulator)** — Input form + real-time results + save estimate + share link (the core interaction)
- **Regional Comparison (/regions)** — Choropleth map + table + sort/filter for the 17 provinces

- **Generation Data** (</data>) — Monthly generation pattern, national annual PV generation trend, Jeju curtailment trend (public statistics)
- **Saved Estimates** (</saved>) — localStorage estimate list, comparison, delete
- **Methodology** (</about>) — Formulas, assumptions, data sources, disclaimer

Every page has a shared **navigation bar** (current page highlighted, hamburger menu on mobile) at the top and a **footer** (disclaimer, key links) at the bottom.

7. Technical Requirements

- **Framework:** Next.js 15 (App Router) + React 19
- **Language:** JavaScript (prioritizing explainability; no TypeScript)
- **Styling:** CSS Modules + global CSS design tokens (no external UI library)
- **State / persistence:** React [useState/useMemo](#); persistent storage via browser [localStorage](#)
- **Backend:** None (all calculations run on the client; no cost or key management)
- **Deployment:** Vercel (automatic deployment via GitHub integration)
- **Structure:** Calculation logic ([lib/calc.js](#)), data ([lib/regions.js](#), [lib/solarHistory.js](#)), and storage ([lib/storage.js](#)) are separated from the UI for reuse and testability

8. Design Requirements

- **Concept:** Warm **amber/orange** accents evoking sunlight, **navy** text conveying trust, and a bright, clean background.
- **Layout:** Centered with a max width of 1080px, card-based information blocks, generous whitespace.
- **Navigation:** Sticky top bar with the current page highlighted; hamburger toggle on mobile.
- **Responsive:** Multi-column grid on desktop that collapses naturally to a single column on mobile.
- **UX principles:** (1) Reflect results immediately on input (minimize feedback delay) (2) Clearly note that values are estimates (3) Surface key metrics in emphasized cards first (4) Usable instantly without sign-up.

9. Milestones

Stage	Deliverable	Date
M1. Planning	Service definition · PRD writing	6/13
M2. Prototype	Next.js structure · calculation engine · simulator	6/13
M3. Feature completion	Regional comparison · saved estimates · methodology page	6/14

Stage	Deliverable	Date
M4. Verification	Build/behavior check · calculation verification · responsive check	6/14
M5. Deployment	GitHub upload · Vercel deployment · README	6/14

AI-Assisted Development Report

1. AI Tools Used

- **Claude (Anthropic, Claude Code)** — used throughout for idea refinement, code generation, debugging, and documentation.

2. Tasks Asked of AI

- Refining the web-service idea for solar installers and drafting the PRD
- Designing the Next.js (App Router) project structure and generating page/component code
- Implementing standard formulas for solar generation, revenue, and payback
- Building the dataset of average daily generation hours for the 17 provinces/cities
- Implementing localStorage-based estimate saving/comparison
- Design-token-based CSS styling and responsive handling
- Building the choropleth map, the generation-data page, and the shareable-link feature
- Debugging build/hydration errors and writing the README and documentation

3. Representative Prompts (3 or more)

1. "I want to build a web service for solar installers using Next.js (App Router). It must run with localStorage only, no backend or paid API. Design a simulator as the core feature: select a region and enter system capacity to calculate annual generation, revenue, and payback period."
2. "Create a dataset of average daily generation hours (equivalent full-load hours) for all 17 provinces/metropolitan cities, and a page that compares annual generation by region. It should support region-group filtering, name search, generation sorting, and a bar-chart visualization."
3. "Save simulator results to localStorage and add a page that compares saved estimates as cards. Automatically highlight the highest-revenue and shortest-payback estimates, and make sure there are no hydration errors caused by server/client rendering differences."
4. "Create a 'methodology' page that transparently explains the generation/revenue formulas and default assumptions, and add disclaimer text in the footer and body noting that the results are estimates."
5. "I want to add a map of South Korea's 17 provinces to the regional comparison page. It must work without an external map API. Convert a public administrative-boundary GeoJSON into simplified SVG paths at build time, color them by generation potential, and let hover/click highlight a region and sync with the comparison table."
6. "Using Korean public statistics (KMA irradiance normals, KPX/Korea Energy Agency generation statistics), build a 'generation data' page that shows the regional monthly generation pattern and the national annual PV generation trend. Draw the charts with SVG (no chart library) and label values as estimates with their sources."

7. "I want a feature that justifies being a web app rather than a spreadsheet. Encode the simulator's inputs into a URL with a 'copy result link' button, and when that link is opened, prefill the form with the same inputs."

4. Parts of the AI Output That Were Modified / Improved

- **Formula verification:** Verified the AI-generated generation/revenue formulas by hand (e.g., $100 \text{ kW} \times 3.8 \text{ h} \times 365 \times 0.8 = 110,960 \text{ kWh}$) and clarified units so REC revenue is computed in MWh.
- **Data realism:** Adjusted regional generation-hour values to match the real tendency (higher in the south — Jeollanam-do, Gyeongsangbuk-do — and lower in Jeju and the capital area), and repeatedly noted "estimate" in the data comments and disclaimer.
- **Grid-constraint modeling (curtailment):** The initial model assumed all generation is sold at SMP, which overestimates revenue in regions like Jeju where curtailment is frequent. Introduced a regional "curtailment rate" and changed the formula to sellable generation = theoretical generation $\times$ (1 - curtailment rate) to adjust revenue and CO₂.
- **Input safety:** Added a safe parser (`num()`) in `lib/calc.js` to handle numeric inputs arriving as strings or as negatives, falling back to defaults.
- **Code structure:** Refactored calculation/data/storage logic into `lib/` so the simulator and the regional comparison page reuse the same logic.
- **Map data reduction:** The original ~7.5 MB province-boundary GeoJSON was too large to bundle, so a Python script (`scripts/build_map.py`) applies equidistant projection + Douglas–Peucker simplification + small-island removal to reduce it to ~32 KB of SVG paths.
- **Information-focused home redesign:** The initial home page had marketing copy and a user-quote (testimonial) section; these were removed to focus on information, restructuring the page around service definition, features, how-to-use, and data sources.
- **Data-source cleanup:** Initially considered overseas (NASA) data, but for credibility in a Korean-installer context, switched to representative values referenced from Korean public statistics (KMA, Korea Energy Agency, KPX) and labeled the source and "approximate" limitation on every screen.
- **Curtailment-data reinforcement:** Referenced KPX non-central output-control information and published statistics (e.g., 181 Jeju curtailment events in 2023) to ground the Jeju/Honam curtailment-rate defaults, and added a "Jeju curtailment trend" chart with its source to the generation-data page. (No public API provides a per-province curtailment rate directly, and the fact that curtailment is effectively concentrated in Jeju is also stated.)

5. Bugs, Errors, Limitations & Fixes

- **Hydration mismatch:** Reading `localStorage` during render on the `/saved` page caused server/client output to differ. → Fixed by reading the data in `useEffect` and using a `mounted` flag to align the first render.
- **Default region selection:** Set Jeollanam-do (highest generation) as the default so the simulator shows a meaningful result on first load.

- **Map projection distortion:** Initially the projection bounds included far islands (Dokdo, Ulleungdo), stretching the map eastward and distorting the mainland. → Fixed by computing the projection bounds only from retained regions above an area threshold.
- **Shareable links and `useSearchParams`:** In the Next.js App Router, `useSearchParams` throws during static build without a Suspense boundary. → Switched to parsing `window.location.search` directly in `useEffect` to work without a build error.
- **Data accuracy limitation:** Because the figures are representative/approximate values referenced from public statistics, they may differ from actual values depending on the source and year. → Each dataset notes its source and the "approximate" nature, and users are directed to original sources (data.go.kr, KMA open data portal, KOSIS) for precise analysis.

6. Final Vercel Deployment URL

<https://assignment4-51gw.vercel.app>